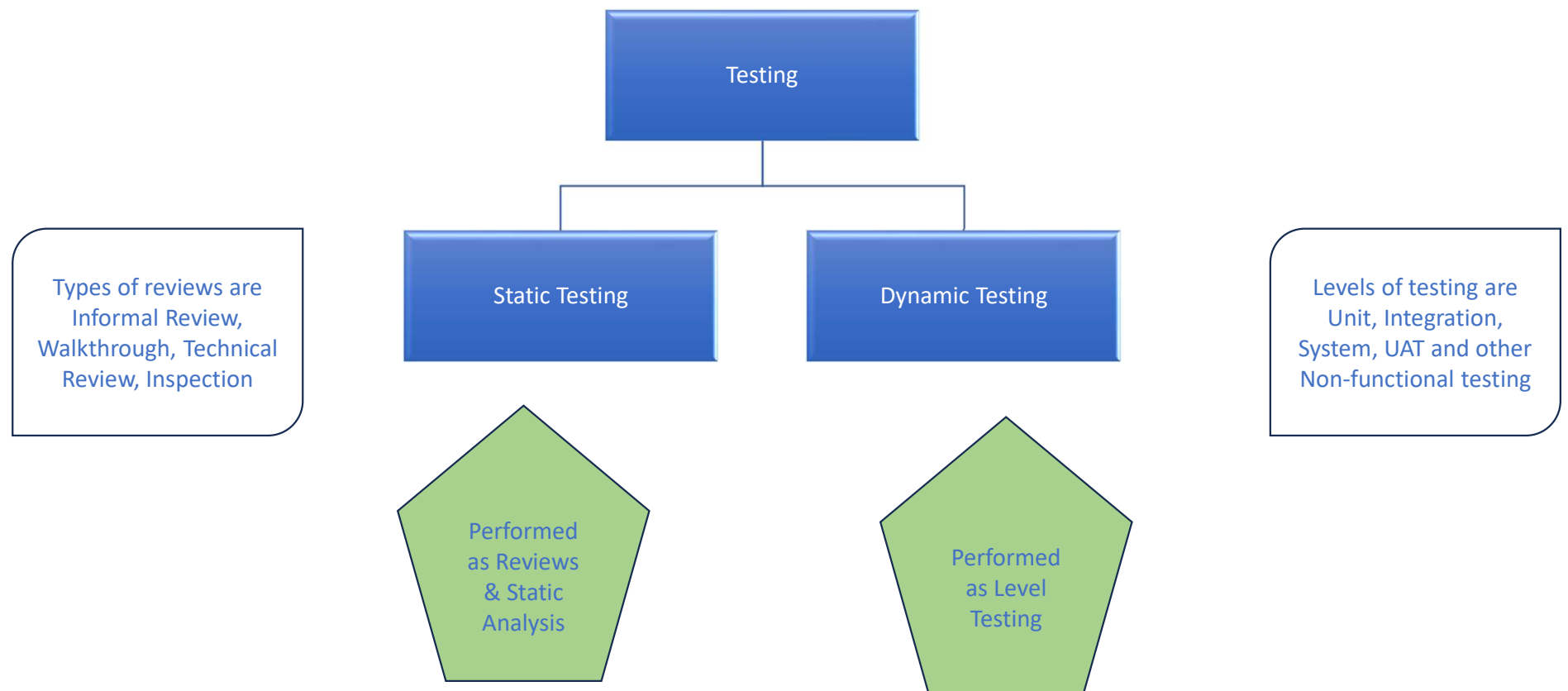


Software Testing & Fundamentals

Module 01: Testing Overview

Classifications of Software testing



Debugging and Testing

- Testing

Testing deal with finding the defects by conducting failure on the application or product. This activity is performed by Testers.

- Debugging

Debugging is a development activity which deals with analyzing these defects, finding the root cause and removes the cause of defects. This activity is commonly performed by developers.

Testing's Contribution to Success

- Using appropriate test techniques are applied with the appropriate level of test expertise, in the appropriate test levels, and at the appropriate points in the software development lifecycle.
- Having testers involved in requirements reviews or user story refinement could detect defects in these work products.
- Having testers work closely with system designers while the system is being designed can increase each party's understanding of the design and how to test it.
- Having testers work closely with developers while the code is under development can increase each party's understanding of the code and how to test it.
- Having testers verify and validate the software prior to release can detect failures that might otherwise have been missed and support the process of removing the defects that caused the failures (i.e., debugging).

Quality Assurance and Testing

- While people often use the phrase quality assurance (or just QA) to refer to testing, quality assurance and testing are not the same, but they are related.
- Quality assurance is typically focused on adherence to proper processes, in order to provide confidence that the appropriate levels of quality will be achieved. When processes are carried out properly, the work products created by those processes are generally of higher quality, which contributes to defect prevention.
- Quality control involves various activities, including test activities, that support the achievement of appropriate levels of quality. Test activities are part of the overall software development or maintenance process.

Error, Defect & Failures

- Error(Mistake)

A human action that produces an incorrect result.

- Fault(Defect, Bug)

A manifestation of an error in software also known as a defect or bug. if executed, a fault may cause a failure.

- Failure

Deviation of the software from its expected delivery or service.

Causes of Software Defects

- Errors may occur for many reasons, such as:
 - Time pressure
 - Human is Error prone
 - Inexperienced or insufficiently skilled project participants
 - Miscommunication between project participants, including miscommunication about requirements and design
 - Complexity of the code, design, architecture, the underlying problem to be solved, and/or the technologies used
 - Misunderstandings about intra-system and inter-system interfaces, especially when such intrasystem and inter-system interactions are large in number
 - New, unfamiliar technologies.

Test Process

- A test process consists of the following main activities
 - Test planning
 - Test monitoring and control
 - Test analysis
 - Test design
 - Test implementation
 - Test execution
 - Test completion

Human Psychology and Testing

- An element of human psychology called confirmation bias can make it difficult to accept information that disagrees with currently held beliefs. For example, since developers expect their code to be correct, they have a confirmation bias that makes it difficult to accept that the code is incorrect.
- Further, it is a common human trait to blame the bearer of bad news, and information produced by testing often contains bad news.
- As a result of these psychological factors, some people may perceive testing as a destructive activity, even though it contributes greatly to project progress and product quality
- This way, tensions between the testers and the analysts, product owners, designers, and developers can be reduced. This applies during both static and dynamic testing.

Psychology of Testing

Testers and test managers need to have good interpersonal skills to be able to communicate effectively about defects, failures, test results, test progress, and risks, and to build positive relationships with colleagues. Ways to communicate well include the following examples:

- Start with collaboration rather than battles
- Remind everyone of the common goal of better quality systems.
- Communicate findings on the product in a neutral, fact-focused way without criticizing the person who created it.
- Try to understand how the other person feels and why they react as they do.
- Confirm that the other person has understood what you have said and vice versa.

Tester's and Developer's Mindset

- Developers and testers often think differently. There are different sets of objectives for them which require different mindsets.
- A mindset reflects an individual's assumptions and preferred methods for decision making and problem solving. A tester's mindset should include curiosity, professional pessimism, a critical eye, attention to detail, and a motivation for good and positive communications and relationships.
- A developer's mindset may include some of the elements of a tester's mindset, but successful developers are often more interested in designing and building solutions than in contemplating what might be wrong with those solutions. In addition, confirmation bias makes it difficult to find mistakes in their own work.
- With the right mindset, developers are able to test their own code.
- Having some of the test activities done by independent testers increases defect detection effectiveness, which is particularly important for large, complex, or safety-critical systems.

Quiz !

Which of the following is a major objective of testing?

Answer Set:

- A. To prevent defects.
- B. To validate the project plan works as required.
- C. To gain confidence in the development team.
- D. To make release decisions for the system under test.

Answer A

Quiz !

Which of the following is an example of a failure in a car cruise control system?

Answer Set:

- A. The developer of the system forgot to rename variables after a cut-and-paste operation.
- B. Unnecessary code that sounds an alarm when reversing was included in the system.
- C. The system stops maintaining a set speed when the radio volume is increased or decreased.
- D. The design specification for the system wrongly states speeds in km/h.

Answer C

Quiz !

Which of the following qualities is MORE likely to be found in a tester's mindset rather than in a developer's?

Answer Set:

- A. Experience on which to base their efforts.
- B. Ability to see what might go wrong.
- C. Good communication with team members.
- D. Attention to detail.

Answer B

PDCA and Workbench Model

A process is the set of activities that is performed to produce a product. Quality is achieved through processes. Processes have the advantage of being able to replicate a product time and time again.

Ways to visually portray a process

- ❑ **PDCA cycle**

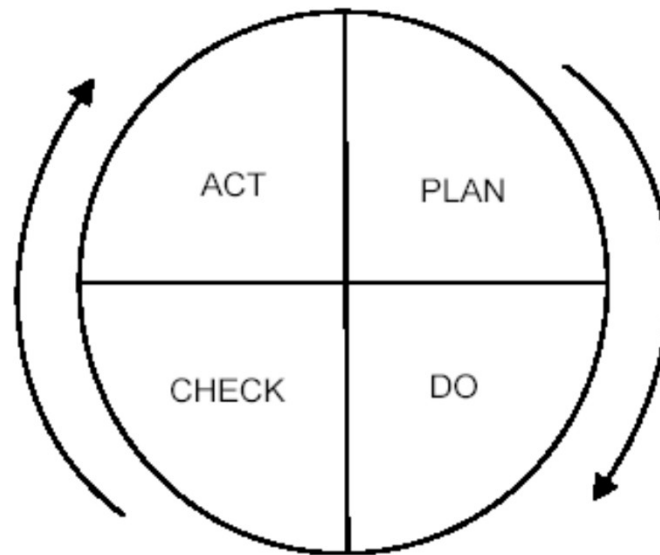
- Conceptual view of a process

- ❑ **Workbench**

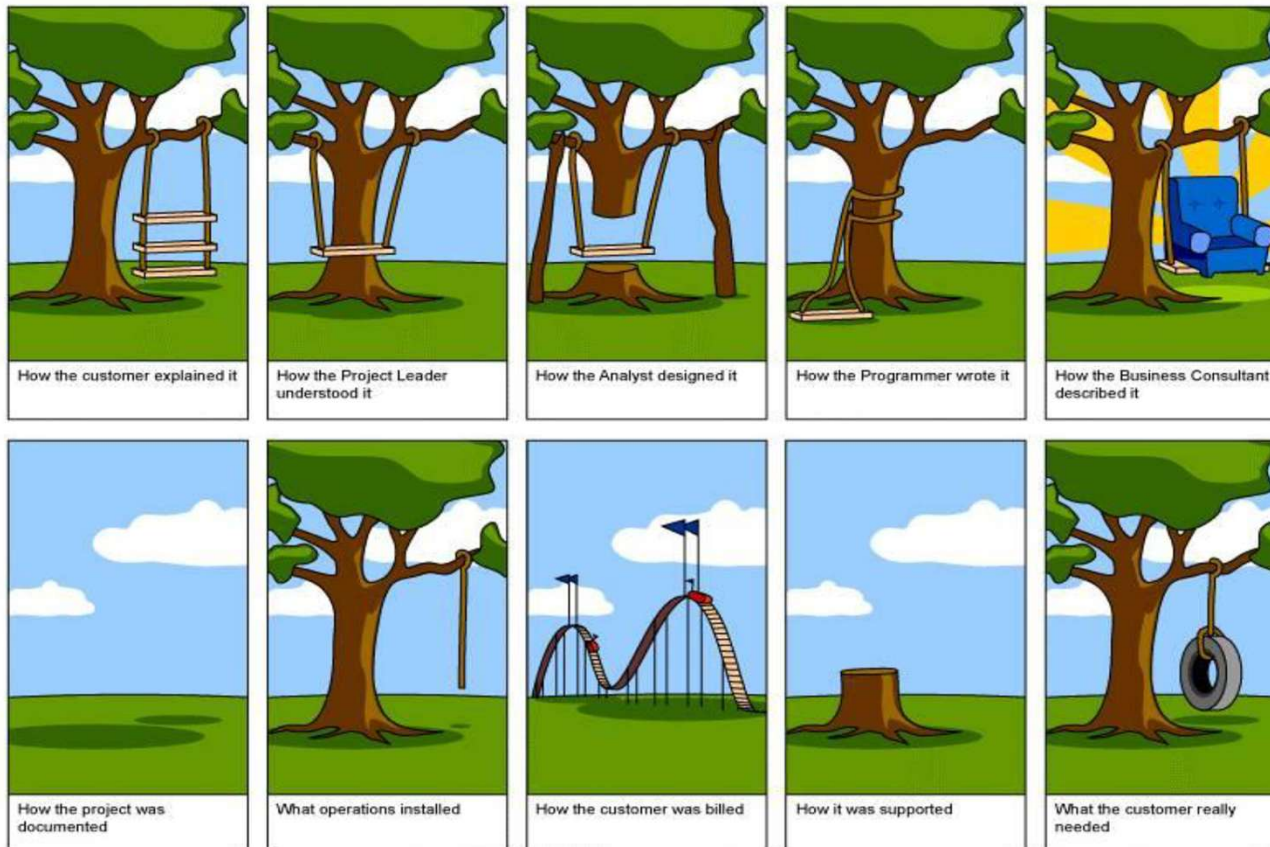
- Practical illustration

PDCA Cycle

- ❑ P – Devise a Plan
- ❑ D – Do the plan
- ❑ C – Check the results
- ❑ A – Act (take any necessary actions)



World of Software Development ...!



What are Defects, Failures, Errors and Incidents?

- **Defect**
 - An imperfection or deficiency in a work product where it does not meet its requirements or specifications.
- **Failure**
 - An event in which a component or system does not perform a required function within specified limits.
- **Error**
 - A human action that produces an incorrect result.
- **Incident**
 - An event occurring that requires investigation.

What is Bug Then???

Software Testing - Objectives

Primary Objectives of Testing

- Finding Defects
- Gaining confidence about the level of quality
- Providing information for decision making
- Prevention of defects

Objectives can also vary based on the kind of Testing

- Development Testing – Find as many defects as possible
- User Acceptance Testing – Confirm system works as expected
- Maintenance Testing – Ensuring no new defects has been introduced
- Operational Testing – Test for reliability and performance

Why do we test?

- Provide confidence in the system
- Identify areas of weakness
- Establish the degree of quality
- Establish the extent that the requirements have been met, i.e. what the users asked for is what they got not what someone else thought they wanted
- To provide an understanding of the overall system
- To prove it is both usable and operable
- To provide sufficient information to allow an objective decision on applicability to deploy

Who will Test ?

- ☐ Users: those who will ultimately use the system
- ☐ Customers: those who define the system – are they the same as the users?
- ☐ Testers?
- ☐ Analysts?
- ☐ Developers?
- ☐ All?

Limitations of Software testing

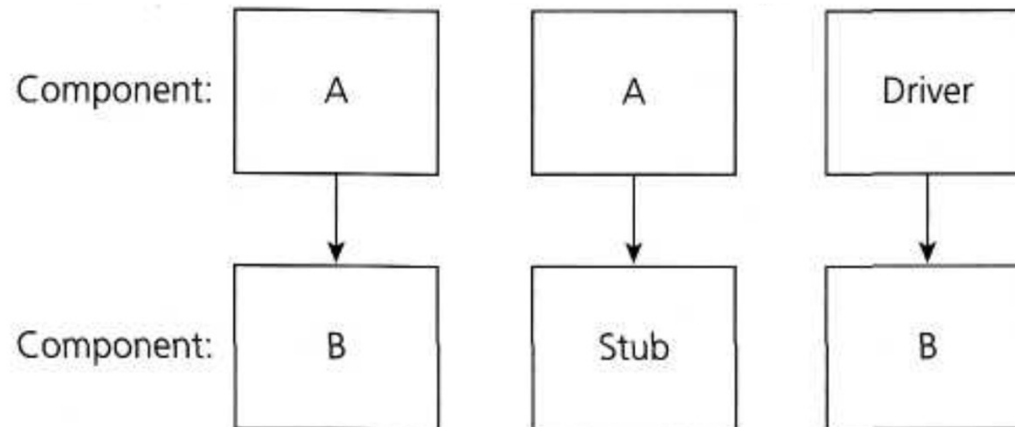
Broadly, the limitations of testing are:

- Testing cannot be exhaustive
- Selective testing cannot detect all errors
- Testing itself may have bugs
- Testing cannot prevent errors; it only detects them
- Testing is not as cost-effective as other quality filters such as formal technical reviews because it comes late in the cycle when errors are more in number and costlier to rectify

Module 02: Testing Levels

Testing Levels – Component Testing

- ❑ Also known as unit, module and program testing
- ❑ It is the testing of an individual unit of software, typically by its developer or by a peer programmer.
- ❑ Identify defects that are not easily identified during other kinds of testing.
- ❑ Enable the developer to confidently iterate and modify the software unit, knowing that any defect will be quickly discovered during regression testing



Testing Levels – Integration Testing

- ❑ Integration testing tests interfaces between components
- ❑ Has to varying approaches:
 - ✓ Big Bang
 - ❖ All modules are integrated and tested for interface compatibility at the same time
 - ✓ Incremental Testing
 - ❖ Each unit is included one at a time, integrated and tested
 - ❖ Includes the following types
 - top-down approach
 - bottom-up approach
 - mixed / hybrid approach

Testing Levels – System Testing

- ❑ System testing is concerned with the behavior of the whole system/product as defined by the scope of a development project or product.
- ❑ It includes tests based on
 - ❖ risks and/or requirements specification
 - ❖ business processes
 - ❖ use cases
 - ❖ high level descriptions of system behavior
 - ❖ interactions with the operating system
 - ❖ system resources

Testing Levels – Acceptance Testing

- ❑ Acceptance Tests determine whether the application satisfies its acceptance criteria.
- ❑ It enable the customer organization to determine whether to accept the application.
- ❑ Results of Acceptance Test can answer questions such as:
 - 'Can the system be released?'
 - 'What, if any, are the outstanding (business) risks?'
 - 'Has development met their obligations?'
 - 'Is the application fit for purpose?'
- ❑ Finding defects is not be the main focus in acceptance testing
- ❑ Is carried out in an environment that closely resembles the real world or production environment

Module 03: Testing Types

Testing Types

- ❑ Different types of testing is carried out to handle specific objectives of testing.
- ❑ A test type could be for a particular test objective such as testing for the:
 - Function of a component
 - Non-functional quality characteristic, such as reliability or usability
 - Structure or architecture of the component or system;
 - Con-firming that defects have been fixed (confirmation testing, or re-testing)
 - Look for unintended changes (regression testing).
- ❑ The formal classification of types of testing are:
 - Testing of function (functional testing)
 - Testing of software product characteristics (non-functional testing)
 - Testing of software structure/architecture (structural testing)
 - Testing related to changes (confirmation and regression testing)

Functional Testing

- ❑ Testing for the function or Functional Testing, checks the system for "what it does"
- ❑ Typically described in
 - ❖ requirements specification
 - ❖ Functional specification
 - ❖ use cases
- ❑ Performed by testers all all test levels
- ❑ Also called as "black-box testing"

Non-Functional Testing

- ❑ Tests for the quality characteristics or non-functional attributes of the System.
- ❑ Helps to validate how well or how fast something is done.
- ❑ Performed at all levels and Includes testing for:

- Reliability
- Usability
- Maintainability
- Availability
- Portability
- Recovery
- Mobile
- Performance
- Security
- Interoperability
- Compatibility
- Installability
- Usability
- Cloud

Module 03: Test Scenario & Test Case Preparation

Test Scenario

- ❑ A test scenario summarizes the functionality to be tested without providing the steps to be followed to test the functionality mentioned in the Use case / Functional Specification
- ❑ Test scenario helps in keeping test focused.
- ❑ Test scenarios are created at a very early stage in the development cycle.
- ❑ Creation of these scenarios requires considerable brain-storming and hence helps the tester in understanding the functionality and identifying the flows that need to be tested
 - e.g.: The actor chooses to withdraw money from the savings bank account.
- ❑ **Inputs for Writing Test Scenario's are :**
 - Business Requirement / Functional Specification Document / Use case
 - Prototype
 - High Level Design Document
 - Test Plan Document

A Test Case

- Test case can be defined as follows:
 - ❑ “A set of test inputs, execution conditions, and expected results developed for a particular objective, such as to exercise a particular program path or to verify compliance with a specific requirement.” (IEEE Standard 610 (1990))
 - ❑ “A sequence of one or more subtests executed as a sequence because the outcome and/or final state of one subtest is the input and/or initial state of the next. The word ‘test’ is used to include subtests, tests proper, and test suites. (Boris Beizer (1995))
- Some of the characteristics of a good test case are as follows:
 - ❑ Coverage – A test case must ensure complete coverage of the software
 - ❑ Modularity - A particular feature of software should not be tested by more than one test case repeatedly
 - ❑ Traceability - The test cases should be traceable to customer requirements.

Test Case - Format

❑ The various ways to represent a test case are as follows:

- Step-by-step or word/action instructions
- Table, matrix
- Script for record/playback or performance test

❑ All need the same structure (anatomy)

Test Case Format - Step-by-step actions

- Usage : Step by step case can be used if unique transactions or rules exist.

Test Step	Test Data / Test Input	Expected Results
Step 1 : Invoke the Internet Explorer and type in the URL name	http://mail.yahoo.com	The yahoo mail homepage should be displayed
Step 2: In the username, enter a valid value:	abc@yahoo.com	The field should accept the User name
Step 3: In the password field, enter a valid value:	xyz	The field should accept the pwd
Step 4 : Click on Sign in button	Select / activate Sign in Button	The user should be logged in and The Yahoo mail page should be displayed

Test Case Format – Table / Matrix

Date	Hired After 1/96	401K	Life Insurance	Payment Computation
10/25/99	Y	1	3	\$24.50
1/4/98	Y	3	1	\$34.00
3/6/96	N	2	5	\$48.00
8/15/96	Y	2	5	\$86.25
8/15/96	N	2	5	\$105.00

Test Case Format - Script for record/playback

```
# Open the Fax Form
menu_select_item ("File;Fax Order...");
set_window("Fax Order");

# Retrieve the Fax Order information and compare it to data from
the main window
edit_get_text("Arrival:", text);
if(main_data["arr_time"] != text)
{
    failure_msg = arrival_fr_mismatch;
    result = FAIL;
```

Test Case Template

Discussion ...

What is the best Template for Test Case ?

Definition of Test Execution

- **What is Test Execution?**
 - Test execution is the phase held after test planning and test preparation that involves a set of required deliverables.
 - Test execution is conducted with either manual or automated test scripts.

Definition of Test Execution

- **Test Execution Objective:**

- The main objective of test execution is to demonstrate that the actual test results for each test step match the expected test results.
- Alternatively, test execution may identify that the system configuration does not meet requirements, which causes the test team to log defects and fix them, thereby increasing the quality of the product.

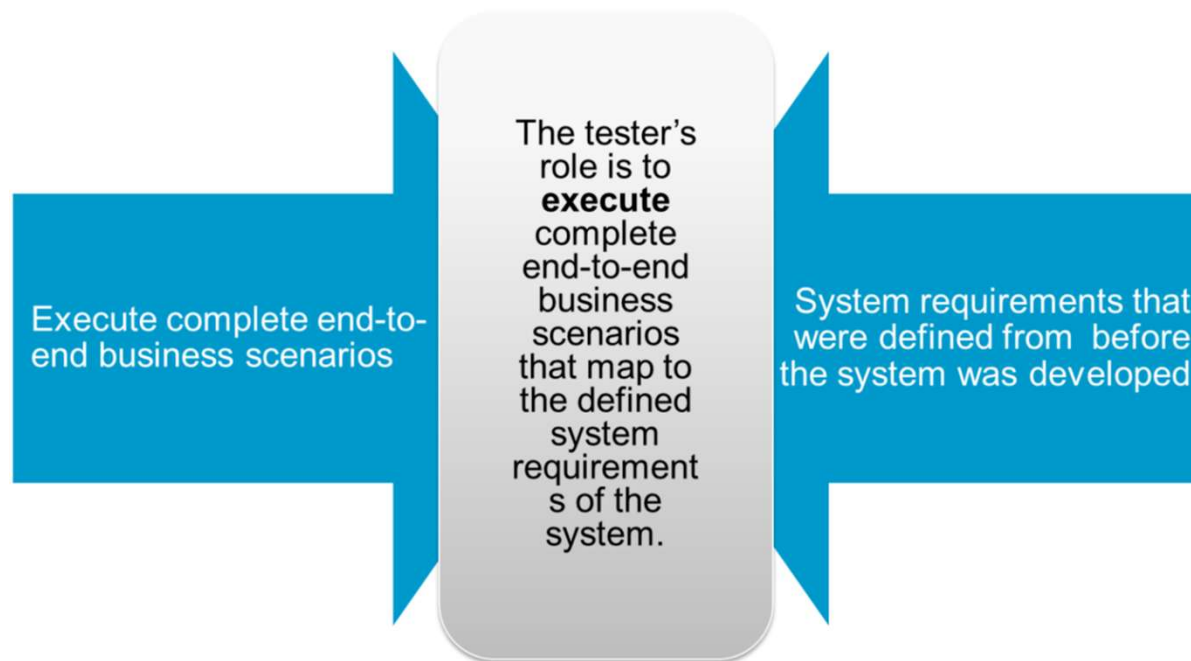
Definition of Test Execution

- **A Developer's Perspective:**

- Testers and Developers work closely together. What do you think a developer will be concerned about?
 - Whether all the business requirements have been properly implemented.
 - If end users are able to successfully execute end-to-end business scenarios.
 - If the system is fully integrated, stable, and operates according to requirements.

Definition of Test Execution

- The Tester's Role



Definition of Test Execution

- **Primary Tester Responsibilities:**

- Execute test scripts.
- Record potential defects.
- Work with the application team to resolve identified defects.
- Participate in the release control process to ensure that solutions meet business requirements.
- Validate application fixes.
- Inform the test lead of any issues that may affect the schedule, budget, or quality of the application or testing process.

Who Should Execute the Test?

Responsible Group	Unit	Integration	System	Acceptance
Testers		X	X	X
Developers	X	X	X	
End- Users			X	X

Module 06: Defects

Where Defects Originate

To understand the dynamics and costs of defects, we need to know some things about them. One of the most commonly understood facts about defects is that most defects originate in the requirements definition phase of a project. The next runner-up is the design phase.

Some problems in getting accurate, clear, and testable requirements are:

- Many people do not have a solid requirements gathering process
- Few people have been trained in or understand the dynamics of requirements
- Projects, people, and the world around us change very quickly
- The English language is ambiguous and even what we consider clear language can be interpreted differently by different people.

Defect Discovery

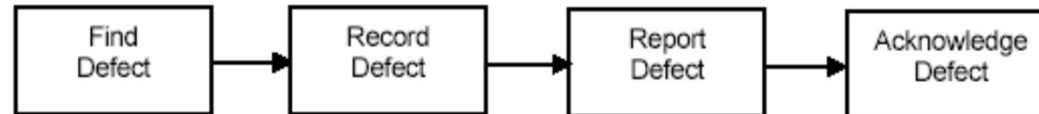
The steps involved in defect discovery are as follows:

- Find Defect
- Record Defect
- Report Defect
- Acknowledge Defect

Defect Discovery

Find a defect

- Static techniques
 - Reviews, Walkthrough, and Inspections
 - Usually the most efficient in finding defects and less costly
- Dynamic techniques
 - Testing!
- Operational techniques
 - Running into a system failure -> defect



Defect Discovery

Record Defect

- To correct the defect
- To report status of the application
- To gather statistics used to develop defect expectations in future applications
- To improve the software development process

Severity vs. Priority

- How bad the defect is vs. How soon should it be addressed
- Severity rules should be set early in the project than used throughout consistently

Report Defects

- If found in test – usually easy to get information to developers
- If found in the field, hope you get enough information

Acknowledge Defect

- Developers need to acknowledge defects quickly and not try to brush off

Ways to Help

- Instrument the code to trap the state of the environment when anomalous conditions occur
- Write code to check the validity of the system
- Analyze reported defects to discover the cause of a defect

A resolution process is needed

The Defect Life Cycle

Discussion ...

What are the Different Status of Defects?

Template for Defect Report.

Module 07: BlackBox Techniques

Test Case Design – Need & Types

- Need for Test Design:
 - Just as code is designed and developed, test cases too must be designed and then written
 - Exhaustive testing of any non-trivial system is impractical.
 - Required to derive at an optimal test suite which is of reasonable size
 - Randomly selecting or writing test cases does not indicate effectiveness of the testing.
 - Writing a large number of test cases does not mean that many errors in the system would be uncovered.
- Types of Test Case Design Techniques
 - Black Box or Specification based techniques
 - White Box or Structure based testing

Black Box Testing – Techniques

- Equivalence class Partitioning
- Boundary Value Analysis
- Use Case Testing
- Error-Guessing

Techniques - Equivalence Class Partitioning

- Input values to a program are partitioned into equivalence classes
- Partitioning is done such that the program behaves in a similar way to every input value belonging to an equivalence class.
- Testing the code with just one representative value from each equivalence class is as good as testing using any other value from the equivalence class
- The equivalence class is determined by examining and analyzing the input data range.

Techniques - Boundary Value Analysis

- Boundaries and conditions are the two major sources of defect in software products
- Typical programming errors occur at the boundaries of equivalence classes
- This may be purely due to psychological factors
- Programmers often fail to see special processing required at the boundaries of equivalence classes
- Programmers may improperly use < instead of <=
- The role of Boundary Value Testing:
 - An additional black box testing approach
 - Test cases at the boundary of each input
 - Includes the following
 - At the boundary
 - Just below the boundary
 - Just above the boundary

Use Case Testing

- Tests can be derived from use cases, which are a specific way of designing interactions with software items, incorporating requirements for the software functions represented by the use cases. Use cases are associated with actors (human users, external hardware, or other components or systems) and subjects (the component or system to which the use case is applied).
- Tests are designed to exercise the defined behaviours (basic, exceptional or alternative, and error handling).
- Coverage can be measured by the percentage of use case behaviours tested divided by the total number of use case behaviours, normally expressed as a percentage.

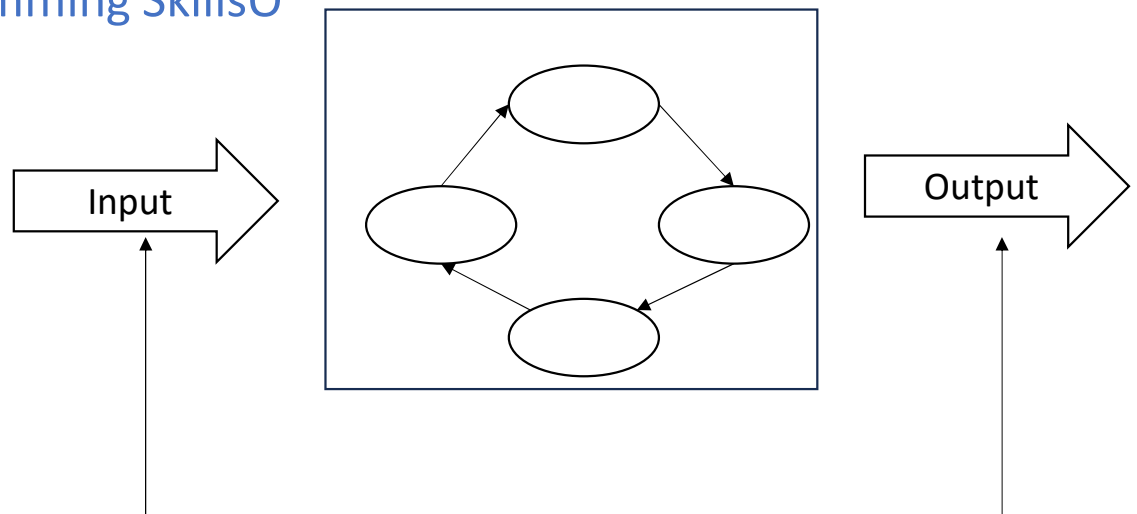
Experience-based Test Techniques

- ❑ When applying experience-based test techniques, the test cases are derived from the tester's skill and intuition, and their experience with similar applications and technologies.
- ❑ Commonly used experience-based techniques are discussed in the following sections.
 - Error Guessing
 - Exploratory Testing
 - Checklist-based Testing

Module 08: Whitebox Techniques

White Box Testing

- White Box Testing
 - Also called Structure Based testing
 - Based on the internal working and structure of the code
 - Requires detailed Programming Skills



White Box Testing – Advantages & Limitations

Advantages

- All independent paths within a module or unit are exercised.
- All logical decisions are exercised on both the true as well as false sides.
- Loops are executed at their boundaries
- Internal data structures are also exercised and tested for their validity.
- As the knowledge of internal coding structure is prerequisite, it becomes very easy to find out which type of input/data can help in testing
- It helps in optimizing the code.
- It helps in removing the extra lines of code, which can bring in hidden defects

Limitations

- All of the execution paths cannot be executed
- Data sensitivity errors may not be detected with the chosen test cases.
- White Box assumes that the control flow is correct.
- The tester must have the programming skills to understand the SUT

White Box Test Techniques

- Statement Coverage
- Condition Coverage
- Decision Coverage
- Function Coverage
- Data Flow Testing

Thank You